Annealed Structured Label Smoothing for Discrete World Models

Florent Tariolle*
INSA Rouen Normandy

Website Code

Abstract

Discrete World Models tokenize observations with a learned quantizer and predict next-frame tokens with a transformer, but standard cross-entropy treats every incorrect prediction as equally wrong. We introduce *Structured Label Smoothing* (SLS), a tokenizer-metric-aware soft-target objective that distributes smoothing mass over nearby tokens instead of uniformly over the vocabulary. Finite Scalar Quantization (FSQ) provides one natural metric through its integer code lattice, while VQ-style tokenizers can use codebook or embedding distance. The current method is *annealed SLS*: structured smoothing is used early to reduce brittle optimization, then the smoothing mass is annealed to zero so the late objective becomes ordinary CE. We evaluate SLS as a minimal objective change on an accepted discrete world-model baseline, and present Geometry Dash as the domain-specific application where FSQ originally exposed the near-miss problem: a real-time Vision-Model-Controller (V-M-C) pipeline trained in imagination, deployed at 30 FPS via screen capture, and able to generate plausible level continuations by rolling its action-conditioned dynamics forward from real gameplay prefixes.

Draft note (2026-06-08). This paper is now framed as a split contribution: annealed SLS as the method, and Geometry Dash V7 as the application. SLS originated from an FSQ-specific observation in Geometry Dash: predicted frames could look visually correct while next-token accuracy stayed low, because neighbouring FSQ codes were penalized like unrelated codes. That remains the motivating special case, but the benchmarkable claim is broader: SLS applies to any discrete tokenizer with a meaningful token metric, and annealing the smoothing mass back to zero avoids the late-training bias of fixed soft targets. The full V7-native Atari controller path is retired for this paper; Atari is used only through accepted-baseline validation. The current IRIS/Pong single-seed result shows the intended pattern: fixed SLS improves the unstable early window but fails asymptotically, while annealed SLS preserves much of the stability/sample-efficiency benefit and recovers near-CE final performance. Additional seeds and at least one more game remain necessary before this becomes a reviewer-grade benchmark result.

1 Introduction

Learning WMs that predict future observations from actions is central to model-based reinforcement learning [1]. Recent approaches tokenize visual frames into discrete codes and train autoregressive transformers to predict token sequences [2, 3]. These models usually optimize hard cross-entropy over tokenizer indices. That objective is indifferent to token geometry: a near-miss token and a semantically unrelated token receive the same penalty whenever neither is the target. This is wasteful when the tokenizer has a meaningful neighbourhood structure, as is common in structured discrete latents.

*Correspondence to florent.tariolle@insa-rouen.fr

Finite Scalar Quantization [4] makes this issue especially visible. Each latent vector is independently rounded to one of L levels per dimension, yielding $\prod_d L_d$ implicit codes with no codebook collapse and no auxiliary losses. Each code maps to a point on a D-dimensional integer lattice, so codes that differ by one step in one dimension often represent visually similar patches. However, the same principle is not restricted to FSQ: any tokenizer with a meaningful token metric, such as VQ codebook or embedding distance, can define a neighbourhood over alternatives.

This places SLS-WM in contrast with diffusion-based generative latent prediction (GLP) methods. Discrete latent spaces have proven effective for sample-efficient Atari world models, most notably in IRIS [2], but discreteness removes the continuous latent geometry that diffusion-style denoising uses to express smooth predictive uncertainty. SLS-WM preserves the discrete classification formulation while recovering a notion of smooth prediction from a token metric: instead of assigning all probability mass to a single token or denoising in continuous space, it distributes target probability over nearby discrete states. In this sense, SLS capitalizes on the discreteness that makes tokenized Atari WMs effective while avoiding the structureless penalty imposed by one-hot supervision.

We propose *Structured Label Smoothing* (SLS), a soft-target objective whose target distribution is a kernel over token distances, so the supervision signal reflects geometry already present in the tokenizer. Classical uniform label smoothing [5] is the structureless limit of this construction and a natural ablation, not the baseline SLS improves over. Our current objective anneals the smoothing mass to zero late in training. The anneal is central: fixed smoothing can stabilize early optimization while biasing the late optimum, whereas annealed SLS becomes exact CE asymptotically. Annealing is the benchmark-clean Atari formulation. In Geometry Dash, fixed SLS can also be interpreted as a domain-specific prior over semantically adjacent FSQ codes, because exact token identity is sometimes stricter than decoded/control-relevant equivalence.

This can be viewed as part of a broader shift from data-agnostic heuristics to structure derived from learned representations, analogous to how learned strided convolutions replaced fixed max-pooling for spatial downsampling [6]. Unlike knowledge distillation, which requires a separate teacher model to produce soft targets, SLS exploits geometry already present in the tokenizer at no additional cost.

Our use of a discrete, reconstruction-anchored tokenizer is deliberate rather than incidental. In real-world video, much of the pixel stream is either unpredictable from the available context or irrelevant to control: lighting variation, sensor noise, sub-pixel texture, camera motion, and unobserved causes can make pixel-level prediction waste compute and even push the representation toward nuisance detail. This matches recent theory comparing reconstruction and joint-embedding SSL: when irrelevant components have large magnitude, latent-space prediction imposes weaker alignment requirements than reconstruction, whereas reconstruction is more competitive when such irrelevant components are weak [7]. For this reason, representation-space objectives such as JEPA are attractive in natural video because they can discard information that should not be predicted. Geometry Dash and Atari are different: they are rendered by deterministic, low-entropy state machines, so the pixels are a repeatable projection of the underlying discrete game state. In this setting, pixel reconstruction provides a useful ground-truth anchor for tokenizing the space, and preserving predictable pixel-level information can help rather than hurt downstream action.

We present SLS in two settings. First, we evaluate annealed SLS as a minimal objective change in an accepted discrete world-model baseline, holding architecture and training budget fixed. Second, we present the Geometry Dash Vision-Model-Controller (V-M-C) pipeline as the application that motivated the method, a fast-paced platformer that demands frame-accurate predictions under a 33 ms latency budget. Our system consists of three components: (1) an FSQ tokenizer that encodes 64×64 Sobel edge maps into 8×8 grids of discrete tokens, (2) a causal transformer that interleaves action tokens with frame tokens in a single sequence and predicts the next frame's tokens via parallel decoding, and (3) a lightweight CNN actor-critic on the predicted token grid, warm-started with Behavioural Cloning and fine-tuned with PPO [8] entirely in dreamed rollouts. Because the dynamics model is action-conditioned and autoregressive, it also acts as a procedural level-continuation generator: seeded with a short observed prefix and a candidate action sequence, it samples future token grids that decode into plausible continuations of the current Geometry Dash segment. The three components are trained sequentially – each on the frozen output of the previous one – and the full pipeline runs in real time at 30 FPS on consumer hardware.

Our contributions are:

1. **Annealed Structured Label Smoothing (SLS)**: a tokenizer-metric-aware training objective that distributes smoothing mass according to token distance early in training, then anneals back to hard CE. FSQ lattice distance is one instantiation; embedding or codebook distance gives the corresponding instantiation for VQ-style tokenizers.
2. **A real-time discrete WM system**: a complete V-M-C pipeline for Geometry Dash running at 30 FPS, combining FSQ tokenization, a transformer with action tokens interleaved in the sequence and an action-conditional contrastive auxiliary loss [9], a CNN actor-critic trained via BC + PPO-in-dreams, and autoregressive generation of plausible level continuations from real prefixes.

2 Related work

Discrete WMs. The Vision-Model-Controller paradigm [1] introduced encoding frames with a VAE, modeling dynamics with an RNN, and training a controller in dreamed rollouts. IRIS [2] replaced the VAE with a discrete tokenizer and an autoregressive transformer, alternating tokenizer and dynamics updates. It provides the accepted-baseline path for CE vs. SLS evaluation, while our Geometry Dash system follows the same broad tokenizer-plus-transformer template with a frozen FSQ tokenizer. DreamerV3 [3] demonstrated that categorical RSSM latents generalize across diverse domains; this approach was scaled further in [10]. TWISTER [9] added action-conditioned contrastive predictive coding to improve temporal coherence; we adopt their AC-CPC auxiliary loss. FSQ [4] is typically positioned as a frozen tokenizer for downstream tasks, and its coordinate-lattice structure has not been exploited at training time. In this paper, FSQ supplies one concrete token metric for SLS, without modifying the tokenizer or its training.

Observation-to-action bootstrapping. Recent autonomous-learning arguments distinguish observation-based learning from action-based learning and emphasize that useful initial structure can help bootstrap later interaction with the environment [11]. This view matches a long-standing practical pattern in RL from demonstrations: AlphaGo first trained policy networks from expert games before reinforcement learning through self-play [12], and DQfD used demonstrations to accelerate Atari reinforcement learning before and during environment interaction [13]. In a closer game-pretraining setting, Video PreTraining learned behavioral priors from human Minecraft videos and then fine-tuned them with imitation and RL for hard-exploration tasks [14]. Our Geometry Dash controller follows the same conservative recipe at smaller scale: expert trajectories provide a short BC warm-start, after which the policy is optimized by PPO in imagined interaction with the learned world model.

Finite Scalar Quantization. FSQ [4] replaces learned codebooks with per-dimension rounding to fixed levels, eliminating codebook collapse and commitment losses. Each code is a point on a D-dimensional integer lattice, giving the codebook a natural coordinate structure. iFSQ [15] introduces a simple modification of the FSQ bounding function that improves bin utilization; we adopt it in our encoder.

Joint-Embedding Predictive Architectures (JEPA). LeWorldModel [16] recently demonstrated a stable end-to-end JEPA WM from pixels using continuous latents and a SIGReg anti-collapse regularizer. We explored a JEPA-style joint-embedding variant of our system, in which context and target frames are encoded by a single online FSQ encoder into a shared discrete latent space and prediction gradients flow back into the encoder through a straight-through path, with the pixel decoder retained only as an anti-collapse regularizer. This was a constrained FSQ-token hybrid, not a faithful LeWorldModel reproduction: canonical SIGReg regularizes continuous Gaussian embeddings, whereas our dynamics model and controller consume bounded FSQ coordinates and hard token IDs. The variant did not improve on the sequentially-trained baseline reported here; we summarize the negative result and the FSQ-compatible anti-collapse surrogate we attempted in Section 5.1. We view this as a domain- and architecture-specific observation rather than as evidence against JEPA: in natural video, avoiding reconstruction of unpredictable pixels is often a feature [7], while in deterministic game domains the rendered pixels are a predictable supervision signal tied closely to the latent state.

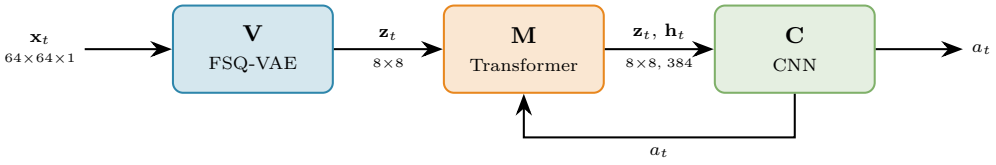

Figure 1: Vision-Model-Controller pipeline. The FSQ-VAE (V) tokenizes Sobel edge maps into a grid of discrete codes, the causal transformer (M) predicts next-frame tokens with action tokens interleaved in the sequence, and the CNN controller (C) reads the predicted token grid alongside the transformer hidden state to select actions. The action feedback loop enables autoregressive dreaming.

Label smoothing. Label smoothing [5] proposed replacing one-hot targets with a uniform mixture to reduce overconfidence. While effective as a regularizer, uniform smoothing ignores class relationships. Knowledge distillation and soft-target methods distribute mass based on teacher predictions, but require a separate teacher model. We instead derive the target distribution from tokenizer geometry itself, requiring no additional model, and anneal the smoothing mass away so the late objective returns to CE.

Adaptive conditioning. FiLM [17] introduced adaptive affine conditioning via scale and shift. DiT [18] applied this as AdaLN-Zero with zero-initialized gates for stable training of diffusion transformers, and QK-norm [19] (RMSNorm on queries and keys per head) was introduced to stabilize attention logits in deep AdaLN models. LeWorldModel [16] applied AdaLN to world-model transformers. Our baseline keeps actions as tokens in the sequence; we treat AdaLN-Zero conditioning as an ablation (Section 5.1).

3 Method

Our system follows the Vision-Model-Controller (V-M-C) paradigm [1] (Figure 1). We describe each component below, with Structured Label Smoothing (Section 3.2) as the primary contribution, integrated into a complete real-time pipeline. The three components are trained sequentially – each on the frozen output of the previous one – a deliberate choice motivated by the ablation results in Section 5.1.

3.1 FSQ tokenization (Vision)

We encode 64×64 grayscale Sobel edge maps using an FSQ-VAE with levels $[8, 5, 5, 5]$, producing 1000 implicit codes. The encoder applies three stride-2 convolutional stages with residual blocks, projecting to a 4-channel latent map of size 8×8. Each spatial position is independently quantized using the iFSQ bounding function [15], $2\sigma(1.6z) - 1$ in place of $\tanh(z)$, scaled to half-levels and rounded, yielding 64 tokens per frame. The encoder is trained on consecutive frame pairs with a pixel-MSE reconstruction loss and the two geometric regularizers from GRWM [20]: a temporal slowness term that penalizes large $\|\mathbf{z}_e^{t+1} - \mathbf{z}_e^t\|^2$ between adjacent frames, and a uniformity term that spreads pre-quantization features across the FSQ hypercube. The decoder mirrors the encoder with transposed convolutions and is discarded after training.

3.2 Structured Label Smoothing

Motivation. Standard label smoothing [5] replaces a one-hot target $\mathbf{e}_i$ with

$$q_{\text{uniform}}(j \mid i) = (1 - \varepsilon)\,\delta_{ij} + \frac{\varepsilon}{V - 1}(1 - \delta_{ij}), \tag{1}$$

where $V = 1000$ is the vocabulary size and ε controls the smoothing strength. This distributes ε uniformly across all alternatives, ignoring codebook geometry.

Token metric. SLS assumes a distance $d(i, j)$ between target token i and alternative token j. For FSQ, a codebook with levels $\mathbf{L} = (L_1, \ldots, L_D)$ assigns each code i a coordinate vector $\mathbf{c}(i) = (c_1(i), \ldots, c_D(i))$ on a D-dimensional integer lattice, giving the squared lattice distance

$$d^2(i, j) = \sum_{d=1}^{D} \left(c_d(i) - c_d(j)\right)^2 . \tag{2}$$

For a learned VQ codebook or an IRIS-style tokenizer, the same construction can use codebook-vector or embedding distance instead. The method is therefore tokenizer-metric-aware rather than FSQ-specific.

Kernel-family study. Given $d(i, j)$, we compare three kernels:

$$k_{\text{Gauss}}(d) = e^{-d^2/2\sigma^2}, \qquad k_{\text{Laplace}}(d) = e^{-d/\sigma}, \qquad k_{\text{Cauchy}}(d) = \frac{1}{1 + d^2/\sigma^2}. \tag{3}$$

The three families differ in tail behaviour. Cauchy has polynomial tails and leaks meaningful mass to distant neighbours; Laplace has a cusp at $d = 0$ and an exponential tail; Gaussian has the thinnest tails, decaying faster than either alternative once d exceeds the bandwidth. We take Gaussian with a small top-k neighbourhood as the current default: thin tails ensure distant neighbours receive negligible mass while the nearest alternatives still receive meaningful target probability.

Metric calibration. A natural alternative is per-dimension weights α_d calibrated from an empirical sensitivity probe: perturbing each FSQ dimension by ± 1, decoding, and measuring reconstruction MSE. The probe gives data-dependent weights, but ties the smoothing target to a particular trained codebook and varies across training runs. For the FSQ Geometry Dash system, we use an isotropic kernel and fix σ by a geometric rule: choose the bandwidth so that the immediate integer lattice neighbour sits near the kernel's half-maximum. For embedding-distance baselines, the corresponding hyperparameters are the smoothing mass, kernel temperature or width, and top-k truncation. The sensitivity probe is retained as an analysis tool to characterize a learned codebook's anisotropy and cross-dimension coupling, but not as a hyperparameter source.

Structured smooth target. We replace the uniform distribution in Equation 1 with a kernel over token distances:

$$q_t(j \mid i) = \begin{cases} 1 - \varepsilon_t & \text{if } j = i, \\ \varepsilon_t \cdot \dfrac{k(d(i, j))}{\sum_{l \neq i} k(d(i, l))} & \text{otherwise.} \end{cases} \tag{4}$$

The full target matrix $\mathbf{Q} \in \mathbb{R}^{V \times V}$ can be precomputed once per kernel choice when the vocabulary is small, or truncated to the top-k neighbours for larger codebooks (Figure 2).

Annealing to CE. The smoothing mass ε_t is scheduled during training. The current benchmark schedule uses $\varepsilon_t = 0.1$ through the early training phase, cosine-anneals from 0.1 to 0 during the middle phase, and then trains with pure CE. This makes the optimization a curriculum: early targets encode neighbourhood structure and reduce brittleness, while the final objective is exactly the hard CE baseline. Fixed SLS is retained as an Atari ablation because it isolates the stabilizing effect but can over-regularize the asymptotic solution. For Geometry Dash, however, we keep SLS fixed rather than annealed: exact token identity is a stricter target than decoded or control-relevant equivalence, and the application is precisely the setting where visually similar FSQ neighbours motivated the method. Thus annealed SLS is the benchmark-clean CE replacement, while fixed SLS is the Geometry Dash instantiation of the same metric-aware prior.

Integration with focal loss. To handle class imbalance (background tokens dominate over obstacle and spike tokens), we combine SLS with focal loss [21]:

$$\mathcal{L}_{\text{SLS-focal}} = -\sum_{j=1}^{V} (1 - p_j)^{\gamma}\, q_t(j \mid i)\, \log p_j, \tag{5}$$

where $p_j = \text{softmax}(\mathbf{z})_j$ and $\gamma = 2.0$.

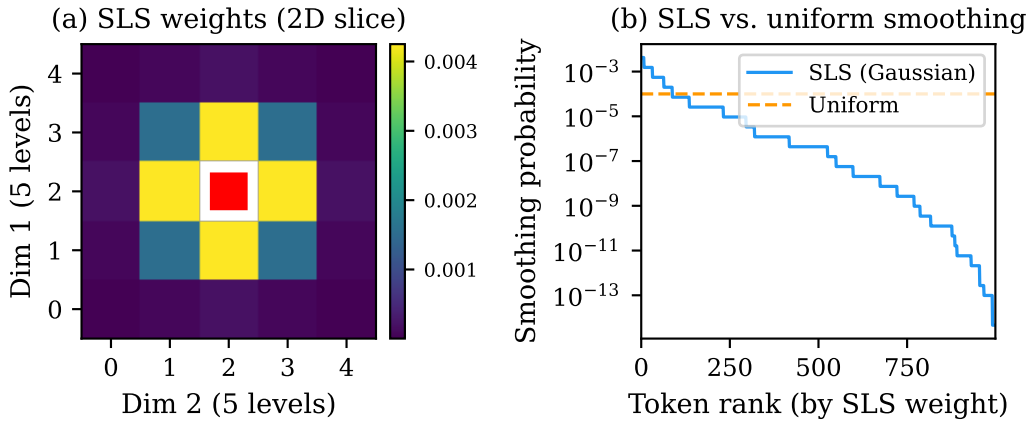

Figure 2: Illustrative SLS target distribution for an FSQ token near the centre of the codebook. **(a)** 2D slice over two lattice dimensions with the remaining two held at the target's values: weights decay as a Gaussian kernel over lattice distance, red square marks the target. **(b)** All $V - 1$ alternative tokens sorted by SLS weight (log scale) vs. uniform label smoothing. SLS concentrates mass on near-miss tokens while suppressing distant ones.

3.3 Transformer WM (Model)

The WM is a causal transformer (384 embedding dim, 8 heads, 8 layers) that predicts the next frame's tokens in parallel given a context of $K = 4$ past frames and their associated actions, on top of a frozen FSQ tokenizer (Section 3.1).

Sequence construction. Each frame contributes 64 visual tokens (from the 8×8 FSQ grid) plus one status token (ALIVE or DEATH), for 65 positions per frame block. Actions are embedded into the same 384-dimensional space and inserted as single tokens between consecutive frame blocks, giving the input sequence

$$[f_0\, a_0\, f_1\, a_1\, \ldots\, f_{K-1}\, a_{K-1}\, f^{\text{tgt}}]$$

of length $K \cdot (65 + 1) + 65 = 329$. All target-frame positions are replaced with a learnable `[MASK]` embedding so the model never sees the ground-truth target tokens during training.

3D Rotary Position Embedding. We apply factored RoPE [22] encoding three positional axes: spatial row, spatial column, and temporal frame index. The head dimension is partitioned into three bands (rows, columns, time) with separate base frequencies ($\theta_{\text{spatial}} = 10$, $\theta_{\text{time}} = 10{,}000$). Status and action tokens receive the grid centre coordinates to represent whole-frame summaries.

Block-causal attention. Within each frame block, attention is bidirectional (tokens attend to all positions in the same frame). Across frame and action blocks, attention is strictly causal: each block attends only to current and past blocks. At inference, the next frame's 64 visual tokens are decoded in a single parallel forward pass from the masked target block.

Action-conditional contrastive auxiliary loss. Following TWISTER [9], we add an action-conditional contrastive predictive coding (AC-CPC) auxiliary loss: for each $k \in \{1, \ldots, K\}$, an MLP head predicts the future hidden state $\mathbf{h}_{t+k}$ from the current $\mathbf{h}_t$ and the intervening actions $a_{t:t+k-1}$, scored against in-batch negatives with InfoNCE. The auxiliary loss enters the total objective with weight $\lambda_{\text{cpc}} = 0.1$.

FSQ-neighbor token noise. On context tokens, with probability 5% per token we replace the token with a lattice neighbour obtained by perturbing a single FSQ dimension by ± 1. An earlier dual-noise scheme also applied uniform random-token replacement, but uniform replacement contradicts the SLS geometry (it presents distant tokens as substitutable for the target, the opposite of the SLS signal on the loss side). We kept only the FSQ-neighbor branch so that context and loss agree on what counts as a near-miss.

Output projection and total loss. The output projection shares weights with the input embedding (weight tying). The total training loss is

$$\mathcal{L} = \mathcal{L}_{\text{SLS-focal}} + \lambda_{\text{cpc}}\, \mathcal{L}_{\text{AC-CPC}}, \tag{6}$$

applied to the transformer parameters with the FSQ tokenizer frozen.

3.4 PPO controller (Controller)

The controller is a lightweight CNN actor-critic, in the spirit of IRIS [2] and DIAMOND, that reads the predicted token grid alongside the transformer's hidden state and outputs a jump probability and a state value estimate.

Architecture. Token IDs are embedded into a small learnable space and reshaped into the 8×8 spatial grid; two stride-1 convolutions with max-pooling produce a 256-dimensional feature vector, which is concatenated with the transformer's hidden state $\mathbf{h}_t \in \mathbb{R}^{384}$ and passed to zero-initialized linear actor and critic heads. The full controller has $\sim$45K parameters.

Training in dreams. The controller is first pretrained via Behavioural Cloning (BC) on expert demonstrations, then fine-tuned with PPO [8] in dreamed rollouts. We use BC as an initialization mechanism rather than as the final learning rule: demonstrations supply a non-random policy from observation, and PPO then performs action-based improvement in the model's imagination [11–14]. At each iteration, several hundred rollouts of up to 45 steps are generated by autoregressively sampling from the frozen WM. Rollouts terminate when the predicted death probability exceeds 0.5. Rewards are $+1$ per survival step with a -0.2 penalty per jump to discourage gratuitous jumping (an idle agent dies immediately upon reaching an obstacle, while a jumping agent survives longer by spending time airborne, creating a deceptive local optimum). We use GAE [8] with $\gamma = 0.995$, $\lambda = 0.95$, and clip ratio $\varepsilon_{\text{PPO}} = 0.2$. A Dreamer-style λ-return actor-critic is a natural replacement: on-policy imagined rollouts make PPO's off-policy clipped surrogate unnecessary. We leave this to future work.

4 Experimental setup

Environment. Geometry Dash is a rhythm-based platformer where the player controls a cube moving at constant horizontal speed. The only action is a binary jump. Colliding with any obstacle causes instant death. Levels are deterministic: the same action sequence always produces the same outcome, but frame-accurate timing is required.

Data collection. We collect roughly 4,200 gameplay episodes ($\sim$250K frames) including both intentional-death episodes (covering diverse failure modes) and 36 expert demonstrations (full level clears). Vertical shift augmentation ($\pm 2, \pm 4$ pixels) expands the dataset $5\times$. Frames are preprocessed with Sobel edge detection and resized to 64×64.

Training details. The FSQ-VAE and the transformer are trained sequentially with AdamW using cosine annealing and warmup. The FSQ-VAE is trained for 200 epochs (LR $4 \cdot 10^{-3}$, batch 32) with the slowness and uniformity regularizers weighted at $\alpha_{\text{slow}} = 0.1$ and $\alpha_{\text{uniform}} = 0.01$, after which the encoder is frozen and its tokens are cached. The transformer is then trained for 200 epochs (LR 10^{-3}, weight decay 0.05, dropout 0.1, batch 256) with 5% FSQ-neighbor token noise on context tokens, and death frames oversampled $15\times$ via weighted sampling. The Geometry Dash transformer uses fixed SLS with smoothing mass 0.1, rather than the Atari annealing schedule, because its role is to preserve a local semantic neighbourhood over FSQ codes rather than to provide a late-stage exact-CE benchmark comparison. BF16 mixed-precision training is used throughout; BF16 requires no loss scaling and is numerically more stable than FP16. Training runs on a single NVIDIA A100 GPU.

Training throughput. We benchmarked `torch.compile` modes under BF16 on our joint-training workload using subprocess isolation to prevent CUDA allocator fragmentation across runs. All `torch.compile` modes are within a few percent of each other; we adopt `reduce-overhead` with `fullgraph=True` as the default: it is faster than `default` mode while avoiding the lengthy autotune phase that `max-autotune` adds at startup in exchange for marginal additional steady-state throughput. This configuration is applied to the world-model and controller training scripts. A secondary

NVIDIA RTX 2060 SUPER (Turing, SM 7.5) was used in parallel for auxiliary experiments. Turing lacks native BF16 (software-emulated and much slower than FP32 on this card) and has no TF32 support; Inductor's channels-last layout heuristic additionally triggers a stride assertion in the convolution backward, which we disable by setting `torch._inductor.config.layout_optimization = False`. Under these constraints the best local configuration is `torch.compile` in `default` mode with FP16 and a `GradScaler`.

Real-time inference. The deployed agent processes each frame at a fixed 30 FPS cadence, with a per-iteration sleep padding any spare time so the capture rate stays constant rather than racing the GPU. Within each iteration we minimize stalls: the captured image is staged into a pinned-memory tensor and uploaded with `non_blocking=True`, so the CPU returns from the H2D copy immediately and queues the subsequent CUDA dispatches (FSQ encode, transformer, controller) without waiting for the transfer to physically complete. The transformer's context window is held as a GPU-resident ring buffer of FSQ token indices, eliminating a GPU→CPU→GPU round-trip per frame. The encoder forward pass is wrapped in a CUDA Graph after a short warm-up, and the entire model graph is compiled with `torch.compile` in `reduce-overhead` mode. Under these optimizations the full pipeline (screen capture, Sobel preprocessing, FSQ encode, transformer step, controller action) measures $\sim$15 ms total per frame on a single RTX 2060 SUPER, well within the 33 ms 30-FPS budget; the agent could in principle run at $\sim$67 FPS, and is configured at 30 FPS for capture-rate stability rather than as a compute ceiling.

BC pretraining ablation. To verify that BC pretraining is a convergence accelerator rather than a stability requirement, we train one PPO run from random initialization. Cold-start PPO converges to the same plateau as BC-initialized PPO on all metrics (eval survival, entropy, jump ratio) but requires roughly 2,000 additional iterations ($\approx$6 hours on A100); BC itself takes $\sim$5 minutes, giving a large compute ROI, consistent with prior demonstration-based RL results where demonstrations mainly improve early learning and sample efficiency [13, 14]. BC is retained in the pipeline on this basis.

Evaluation metrics. We evaluate on: (1) next-token prediction accuracy and cross-entropy on held-out episodes, (2) death prediction F1 score, and (3) real-environment gameplay performance measured as percentage of level completed (mean over $N = 100$ automated runs). For the application section, we also retain qualitative generated rollouts: the WM is seeded with real gameplay frames and action tokens, then rolled forward to visualize level continuations and failure/survival branches in token space.

5 Results

Current benchmark result. The first matched IRIS/Pong seed compares CE, fixed SLS, and annealed SLS under the same architecture, data, training budget, BF16 precision, and `torch.compile` mode. This is not yet a final benchmark table: more seeds and at least one additional game are required before submission. It is nevertheless useful for deciding the paper framing.

Condition	Final return	Best return	Interpretation
CE	20.9375	20.9375	strong final performance, unstable mid-training
Fixed SLS	0.125	low single digits	stabilizes early training but over-regularizes late
Annealed SLS	19.625	19.8125	recovers near-CE final performance with better early/mid stability

On the common CE-vs-annealed-SLS curve, annealed SLS improves the mean return over the unstable epoch 250–420 window by $+7.15625$ and improves eval-return AUC from -993.0208 to -408.90625. CE remains slightly better in the final tail (mean return after epoch 500: 19.6042 for CE vs. 17.0923 for annealed SLS; final return 20.9375 vs. 19.625). This supports a stability/sample-efficiency claim, not a claim that SLS universally improves final return.

Planned result table. The final paper table should report CE vs. annealed SLS over multiple seeds, with return AUC, collapse-window return, rolling variance, first epoch to return thresholds, final return, and tail mean return. Fixed SLS should remain as a diagnostic negative ablation from the first

seed, showing why annealing is part of the method, but it is not a compute priority for replication. This does not contradict the Geometry Dash design choice: there, fixed SLS is supported by the tokenizer-neighbour visualizations and development ablations, but we do not present a fully replicated CE-vs.-fixed-vs.-annealed Geometry Dash benchmark.

5.1 Ablations and negative results

We report a set of architectural changes which, on paper, were expected to improve over the baseline of Section 3, but did not yield real-game gains in our experiments. Together they motivate the deliberately conservative system design of the current paper. Detailed per-ablation tables are pending.

Joint training of FSQ and transformer. We trained a JEPA-style variant in which the FSQ encoder and the transformer are optimized together in a single AdamW group, with prediction gradients routed back into the encoder through a learnable straight-through projection (`fsq_grad_proj`) inserted between the pre-rounding continuous latents and the transformer input. A light pixel-MSE loss is retained as an anti-collapse regularizer on the encoder, and a single global gradient-norm clip is applied to the combined parameter set following [16]. This setup trains stably and produces lower validation cross-entropy than the sequentially-trained baseline, but the resulting FSQ codebook drifts toward a representation that is harder to act on: real-game level progress consistently dropped relative to the sequentially-trained baseline of identical capacity. Removing the pixel-MSE anchor (pure joint embedding) causes the encoder to collapse. We did not run canonical SIGReg in this discrete variant: LeWorldModel regularizes continuous embeddings toward an isotropic Gaussian, while our tokenizer exposes bounded FSQ coordinates and hard token IDs. Instead, we tried an FSQ-compatible distribution-matching surrogate inspired by SIGReg, using a Cramér-von-Mises-style test against factorized code utilization; it was insufficient to prevent joint-distribution degeneracy when the prediction target comes from the online encoder. We therefore keep the reconstruction-anchored tokenizer for the reported system. This choice should not be read as a faithful comparison to LeWorldModel or as a claim that pixel prediction is generally preferable to JEPA: in natural videos, forcing a model to spend capacity on unpredictable pixel detail can waste compute and degrade the representation, consistent with theoretical and empirical evidence favoring joint-embedding objectives when irrelevant input components are high-magnitude [7]. The comparison is further shaped by our observation space: the tokenizer reconstructs grayscale Sobel edge maps rather than raw RGB frames, so preprocessing already removes much of the color, texture, and background variation that joint-embedding objectives are often meant to ignore. This makes reconstruction a less wasteful anchor here than it would be from raw natural video, and gives the JEPA-style variant less room to differentiate itself within this constrained architecture. Our claim is narrower: in deterministic discrete games, where pixels are predictable outputs of the simulator and small pixel errors often correspond to state errors, reconstruction is a useful anchor for a token space that the dynamics model and controller can share. Finding a faithful decoder-free discrete anti-collapse signal, or a full continuous-latent LeWorldModel-style Geometry Dash system, remains open work.

AdaLN-Zero action conditioning. Rather than inserting actions as tokens, we conditioned each transformer block on the action history via Adaptive Layer Normalization [18, 17]. The K-step action sequence was projected through an MLP to per-layer scale, shift, and gate parameters modulating each sublayer. To stabilize training we needed both QK-norm [19] (RMSNorm on queries and keys per head) and gate-bias initialization to 1 rather than DiT's standard zero: zero-initialized gates suppressed both conditioning and learning, and without QK-norm, attention logits diverged. Even with both fixes, the AdaLN variant matched but did not surpass the in-sequence-action baseline on real-game level progress, while adding architectural complexity. The token-in-sequence design is what we ship.

Larger transformer ($384 \to 512$). Scaling the embedding dimension from $384 = 384$ to 512 at otherwise matched capacity improves dream-quality metrics (validation cross-entropy and AC-CPC score) but does not translate into real-game gains, with extra training time per epoch. We retain the smaller model.

MLP controller on $\mathbf{h}_t$ alone. We tested a controller that consumes only the transformer hidden state $\mathbf{h}_t$, with no CNN over the predicted token grid, motivated by the fact that $\mathbf{h}_t$ already encodes spatial and temporal structure through 8 transformer layers with 3D-RoPE. This MLP variant converges

2–5× faster in PPO and has the smallest train-eval gap of any controller we trained, but the CNN-on-tokens variant from Section 3.4 dominates it on real-game level progress, suggesting the explicit token-grid signal carries information that is washed out in the global hidden-state summary.

6 Limitations

The current benchmark evidence for annealed SLS is limited to one IRIS/Pong seed and must be expanded before submission. The present result supports the choice of framing and the need for annealing, but not yet a broad claim across Atari games or random seeds. The full V7-native Atari controller path did not become a reliable evaluation vehicle and is explicitly out of scope for this paper. Geometry Dash remains a single-game application with a binary action space and deterministic level layouts; its deploy-survival metric is useful for the system section but too noisy to be the primary SLS effect-size benchmark. The level-generation claim is likewise scoped to autoregressive visual continuations sampled from the learned dynamics model, not to exporting editor-valid Geometry Dash levels with guaranteed playability. The generalization of SLS to environments with higher-dimensional actions, stochastic dynamics, or RGB observations remains to be validated. The system is trained sequentially – FSQ, then transformer, then controller – which is operationally simple but forfeits the possibility of prediction gradients shaping the codebook. Our preliminary attempts at a JEPA-style joint-training variant inside the existing FSQ-token architecture did not produce real-game gains over this sequential baseline (Section 5.1); whether a faithful continuous-latent LeWorldModel adaptation, a more careful version of joint FSQ training, or a stronger decoder-free discrete anti-collapse signal would close this gap is open work. The reconstruction anchor is also domain-specific: it is appropriate here because the rendered observations are predictable projections of a discrete game state, but the same objective may be wasteful or counterproductive in real-world video where much of the pixel information is not predictable from the agent's context.

7 Conclusion

We introduced annealed Structured Label Smoothing, a tokenizer-metric-aware objective for discrete-token world models. SLS replaces hard CE targets with a kernel over nearby tokens early in training, then anneals the smoothing mass to zero so the final objective becomes CE. FSQ lattice distance gives the Geometry Dash instantiation; codebook or embedding distance gives the corresponding instantiation for VQ-style tokenizers. We also integrate SLS into a real-time discrete-WM system on Geometry Dash: an FSQ-VAE tokenizer, a causal transformer with action tokens interleaved in the sequence and a TWISTER-style action-conditional contrastive auxiliary loss, and a CNN actor-critic on the predicted token grid trained via BC and PPO entirely in dreamed rollouts. The same action-conditioned dynamics can be used generatively: from a real gameplay prefix, the model rolls out plausible future token grids and decoded frames, providing procedural level-continuation samples in addition to control-oriented dreams. The three components are trained sequentially, and the full pipeline runs at 30 FPS on consumer hardware. A set of architectural variants (joint FSQ+M training, AdaLN-Zero action conditioning, scaling to a larger transformer, an MLP controller on the hidden state) is reported as ablations: each is more elegant on paper but did not improve real-game level progress in our experiments, motivating the deliberately conservative baseline of this paper. The remaining paper-critical work is to expand the IRIS benchmark beyond the first Pong seed and report stability, sample-efficiency, and final-performance KPIs under matched CE and annealed-SLS conditions, with fixed SLS retained as the negative-control ablation motivating annealing. A natural extension is to make the kernel bandwidth or schedule learnable and jointly optimized with the WM; care must be taken to prevent degenerate solutions in which the learned targets collapse to uniform smoothing or preserve late smoothing after CE should dominate.

Acknowledgments and Disclosure of Funding

We thank Maël Planchot for contributing gameplay data. The NVIDIA A100 GPU used for all primary training runs was provided by MesoNet (CRIANN, Rouen) through the Representation Learning course at INSA Rouen Normandy. Geometry Dash is developed by RobTop Games.

References

[1] David Ha and Jürgen Schmidhuber. World models. In *Neural Information Processing Systems*, 2018.

[2] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023.

[3] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025.

[4] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, 2024.

[5] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the Inception architecture. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

[6] Jost Tobias Springenberg, Alexey Dosovitskiy, Thomas Brox, and Martin Riedmiller. Striving for simplicity: The all convolutional net. In *International Conference on Learning Representations (Workshop)*, 2015.

[7] Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint embedding vs reconstruction: Provable benefits of latent space prediction for self supervised learning. *arXiv preprint arXiv:2505.12477*, 2025.

[8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[9] Maxime Burchi and Radu Timofte. TWISTER: Learning transformer-based world models with contrastive predictive coding. In *International Conference on Learning Representations*, 2025.

[10] Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models. *arXiv preprint arXiv:2509.24527*, 2025.

[11] Emmanuel Dupoux, Yann LeCun, and Jitendra Malik. Why AI systems don't learn and what to do about it: Lessons on autonomous learning from cognitive science. *arXiv preprint arXiv:2603.15381*, 2026.

[12] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Vedavyas Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

[13] Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Gabriel Dulac-Arnold, Ian Osband, John Agapiou, Joel Z. Leibo, and Audrunas Gruslys. Deep q-learning from demonstrations. In *AAAI Conference on Artificial Intelligence*, 2018.

[14] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (VPT): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.

[15] Mohammad Hassan Vali, Tom Bäckström, and Arno Solin. iFSQ: Improving FSQ for image generation with 1 line of code. In *International Conference on Learning Representations*, 2026.

[16] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[17] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence*, 2018.

[18] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *International Conference on Computer Vision*, 2023.

[19] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorber, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning*, 2024.

[20] Zaishuo Xia, Yukuan Lu, Xinyi Li, Yifan Xu, and Yubei Chen. Clone deterministic 3d worlds with geometrically-regularized world models. *arXiv preprint arXiv:2510.26782*, 2025.

[21] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *International Conference on Computer Vision*, pages 2980–2988, 2017.

[22] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

Paper Checklist

1. **Claims**

   Answer: [Yes]

   Justification: The abstract and introduction state the two contributions: annealed SLS as a discrete-token world-model objective, and Geometry Dash V-M-C as the application system. The benchmark scope and current single-seed limitations are stated explicitly.

2. **Limitations**

   Answer: [Yes]

   Justification: Section 6 discusses the current one-seed IRIS/Pong benchmark state, the retired V7-native Atari controller path, Geometry Dash's single-environment scope, binary action space, and tokenizer-metric transferability.

3. **Theory assumptions and proofs**

   Answer: [N/A]

   Justification: The paper does not include theoretical results. SLS is an empirical method.

4. **Experimental result reproducibility**

   Answer: [Yes]

   Justification: Section 4 provides full training details. Code will be released.

5. **Open access to data and code**

   Answer: [Yes]

   Justification: Code and dataset are publicly available at `https://github.com/Tariolle/sls-wm`. Data collection procedure is described in Section 4.

6. **Experimental setting/details**

   Answer: [Yes]

   Justification: All hyperparameters, optimizers, schedules, and data augmentation are specified in Section 4.

7. **Experiment statistical significance**

   Answer: **[TODO]**

   Justification: **[TODO]**

8. **Experiments compute resources**

   Answer: [Yes]

   Justification: All experiments run on a single NVIDIA A100 GPU. Training times are reported in Section 4.

9. **Code of ethics**

   Answer: [Yes]

   Justification: The research involves no human subjects, sensitive data, or dual-use concerns.

10. **Broader impacts**

    Answer: [N/A]

    Justification: This work applies to a single-player video game with no direct societal impact.

11. **Safeguards**

    Answer: [N/A]

    Justification: The released model is a small world model for a video game and poses no misuse risk.

12. **Licenses for existing assets**

    Answer: [Yes]

    Justification: All referenced works are properly cited. Geometry Dash is a commercial game by RobTop Games; gameplay data was collected from a legally purchased copy.

13. **New assets**

    Answer: [Yes]

    Justification: Code and model checkpoints will be released under an open-source license with documentation.

14. **Crowdsourcing and research with human subjects**

    Answer: [N/A]

    Justification: No crowdsourcing or human subjects research was conducted.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Answer: [N/A]

    Justification: No human subjects research was conducted.

16. **Declaration of LLM usage**

    Answer: [N/A]

    Justification: LLMs were not used as a component of the core methodology.